OPTIMIZED NESTING ENVIRONMENT (ONE)

Last Updated: September 29, 2026

The optimized nesting environment (ONE) is a new Ilumbra led format for storing all volumes, textures and scene parameters for the correct display of each 3D volumetric model. The format stores a list of volumes (region in the scene) and their associated textures.

PART 1: THE OPTIMIZED NESTING ENVIRONMENT (.ONE) SPECIFICATION

Object Definitions

Scene: Contains all the textures, volumes and information needed to render a model.
Texture: Non-zero voxels covering a volume in space.
Volume: A distinct region in the scene.
Voxel: An rgba value at a given location in space.

A ONE file is ordered with the data FIRST and then the header. This allows for quick header writing without having to re-read the large textures.

Each ONE object has a list of parameters (described below).

File Format (.ONE)

The following describes the format of the ONE file. Each entry in **bold** indicates the type of variable saved/read. The **string** variable is encoded using a modified-utf-8 format.The first two bytes of the **string** variable represent the number of bytes in the following string, which is encoded in the modified UTF-8 format.The data is stored in Big-Endian format, with an **int** being 4 bytes, a **float** 4 bytes, and a **long** 8 bytes.

In the case of voxels, the data can either be written as **float** or **byte** depending on the texture `TYPE` parameter. If the texture `TYPE` parameter is `RGBA_FLOAT` the voxel data will be **float**, and if the texture TYPE parameter is `RGBA_BYTE` the voxel data will be **byte**.

The parameter **string** read for each ONE object consists of all parameters concatenated by the "!@" string. Each parameter consists of a KEY and VALUE string, separated by a '**:**' character. As an example, the parameter string from the scene object may look like:

"EXPOSURE:15.0!@EMISSION:1.0!@OPACITY:600.0"

In this example, there are 3 parameters: EXPOSURE:15, EMISSION:1.0 and OPACITY: 600.

The steps to read the ONE file should be similar to the following:

1. Seek to the LAST long value of the file. This will indicate the length of the header.
    a. filePos = fileSize - longSize
    b. Seek(filePos)
    c. headerLength = readLong()

2. Seek to the start of the header.
    a. headerStartPos = fileSize - headerLength - longSize;
    b. Seek(headerStartPos)

3. Read the Header
4. Seek to the beginning of the file.
5. Read the texture data. The number of textures to read will be available from the number of textures read from the header.

File Structure

Data

- For each Texture
  - **long** : Texture ID
  - **int** : Number of Voxels
  - For each Voxel:
    - **int** : x index of voxel
    - **int** : y index of voxel
    - **int** : z index of voxel
    - **byte**/**float** : r value
    - **byte**/**float** : g value
    - **byte**/**float** : b value
    - **byte**/**float** : a value

Header

- **int**: ONE File ID = 102380. Used to identify ONE files.
- **int**: ONE version
- Scene
  - **long** : Scene ID
  - **string** : Scene name
  - **string** : Parameter string (see above)

- Volumes
  - **int** : Number of Volumes in the scene
  - For each Volume
    - **long** : Volume ID
    - **string** : Volume name
    - **string** : Parameter string (see above)

- Textures
  - **int** : Number of Textures in the scene
  - For each Texture
    - **long** : Texture ID
    - **string**: Texture name
    - **string**: Parameter string (see above)

- **long** : Header length in bytes

Common Parameters

Below is a list of common parameters to describe how to orient the volumes and render the scene. There is no limit to how many parameters can be included.

ONE SCENE

Parameters	Description
EXPOSURE	The factor by which to multiply the overall rendered pixel values.
EMISSION	The factor by which to multiply the rgb values of each voxel.
OPACITY	The factor by which to multiply the ‘a’ values of each voxel.
ROT_X	The x-rotation (deg) of the model to achieve the ‘from earth’ viewpoint.
ROT_Y	The y-rotation (deg) of the model to achieve the ‘from earth’ viewpoint.
ROT_Z	The z-rotation (deg) of the model to achieve the ‘from earth’ viewpoint.

ONE VOLUME

Parameters	Description
TEXTURE_ID_X	The ID (s) of the texture (s) this volume will render.
ORDER	The order of the volume in the scene. Smaller order volumes are ‘on top’ of higher order volumes.
BLEND	How much to blend the volume with the one ‘above’ it (0-1)
REPLACE	Should this volume replace the volumes above it? (true / false)
EMISSION	The factor by which to multiply the rgb values of each voxel in the volume.
OPACITY	The factor by which to multiply the a values of each voxel in the volume.
SCALE_X	The size of the volume in the x direction with respect to the entire scene.
SCALE_Y	The size of the volume in the y direction with respect to the entire scene.
SCALE_Z	The size of the volume in the z direction with respect to the entire scene.
OFFSET_X	The x-position of the volume with respect to the scene. (-1 to 1)
OFFSET_Y	The y-position of the volume with respect to the scene. (-1 to 1)
OFFSET_Z	The z-position of the volume with respect to the scene. (-1 to 1)

ONE TEXTURE

Parameter	Description
TYPE	The type of data represented in this texture. Values may be one of RGBA_FLOAT or RGBA_BYTE
WIDTH	The width of the texture cube.
HEIGHT	The height of the texture cube.
DEPTH	The depth of the texture cube.
CONTENT_TYPE	The type of content stored in the texture. One of VOLUME, PROCEDURAL, STARS
MAX_GREY	The maximum grey value in the texture. The grey value is computed as (r + g + b) / 3;
MAX_A	The maximum ‘a’ value in the texture.
MIN_X	Index of the first voxel that contains non-zero colors in the x direction.
MIN_Y	Index of the first voxel that contains non-zero colors in the y direction.
MIN_Z	Index of the first voxel that contains non-zero colors in the z direction.
MAX_X	Index of the last voxel that contains non-zero colors in the x direction.
MAX_Y	Index of the last voxel that contains non-zero colors in the y direction.
MAX_Z	Index of the last voxel that contains non-zero colors in the z direction.
J_SCALE	Scaling that should be applied to the rgb values.
K_SCALE	Scaling that should be applied to the ‘a’ values.

PART 2: AUGMENTING VOLUMETRIC NOISE (AVN) PROTOCOL

AVN is a way to add targeted procedural noise detail to an underlying volumetric model. AVN is applied through the rendering of specifically constructed textures delivered within a .ONE scene. AVN textures contain data representing “paths” that snake through a scene providing information about where to place noise. The AVN textures must be decoded and appropriately rendered within the fragment shader of the rendering software. The goal is to transform each ray sample position (x,y,z) to a path coordinate (px,py,pz). Applying procedural noise to the path coordinates allows for the noise to be directed along the paths.

The trick is to determine the path coordinate at each point in space. To do this, AVN divides each path into many short segments. Each segment may either be a straight line, or a curved corner connecting two straight lines. At its core, for each ray sample, the AVN texture is used to look up which path segments are needed to be rendered. The properties for each of these segments are then used to transform the ray sample position to path coordinates.

The AVN texture format is optimized to make path segment look-ups as efficient as possible. Below describes the format and packing of these textures.

A single AVN textures contain multiple 3D layers of size width/height/depth.

- Base Volume Data
  - Layer #: 0
  - Contains the RGB colors of the path emission at the x,y,z voxel. The A value represents the density at this location.
  - No AVN contribution is emitted where A = 0

- Segment Header
  - Layer #: `AVN_SEGMENT_HEADER_LAYER`
  - Each x,y,z voxel contains information about which segments may intersect that region of space.
  - Provides an index into the Candidate-List Layer which contains the actual segment list for this voxel. This allows AVN to have many overlapping segments in one region.
    - R = low 16 bits of candidate-list offset
    - G = high 16 bits of candidate-list offset
    - B = candidate count
    - A = not used
    - Offset into Candidate-List Layer: `offset = R | (G << 16)`
  - This layer contains a list of all POSSIBLE segments in this region. Each segment may or not be rendered depending on its specific radius.

- Candidate-List Layer(s)
  - Layer #: `AVN_CANDIDATE_LIST_LAYER`
  - Flattened sequence of integer segment IDs.
  - `candidate list[offset + i] = segment ID`
  - Because a single logical layer may not be large enough, the list can spill into consecutive layers. Each layer is the same size, width/height/depth

- Segment Table Layer (s)
  - Layer #: `AVN_SEGMENT_TABLE_LAYER`
  - Contains all the needed information for each segment in the AVN texture.
  - Flat fixed-stride record table with stride = `AVN_SEGMENT_RECORD_TEXELS`
  - The start of each segment record is given by:
    - `baseRecord = (segmentID - 1) * AVN_SEGMENT_RECORD_TEXELS`
  - Each segment stores 8 texel records (their interpretation depends on if they are a line or corner segment):

Texel Record Descriptions

Record Texel	Line Description	Corner Description
0	rgb: `p0.xyz` , w: `type`	rgb: `v0.xyz` , w: `type`
1	rgb: `p1.xyz` , w: `pathID`	rgb: `v1.xyz` , w: `pathID`
2	rgb: `xAxis.xyz` , w: `z0`	
3	rgb: `yAxis.xyz` , w: `radius`	
4	rgb: `zAxis.xyz` , w: not used	
5	not used	rgb: `center.xyz` , w: `cornerRadius`
6	not used	rgb: `planeNormal.xyz` , w: `theta`
7	r: `branch0`, g: `branch1`, b: `isLeaf`, a: `pathLength`	

Variable Descriptions

Variable	Description
`p0`	Starting point of the line segment

p1	Ending point of the line segment
v0	Vector from the center of the corner circle to the starting point of the segment.
v1	Vector from the center of the corner circle to the ending point of the segment.
type	Type of segment. 1 = Line segment, 2 = Corner segment
pathID	The ID of the path this segment is a part of (there may be multiple paths)
xAxis	The direction of the path's x axis
yAxis	The direction of the path's y axis
zAxis	The direction of the path's z axis
z0	The length along the entire path of the start of this segment
radius	Radius of the segment
center	Center of the corner arc
planeNormal	The normal direction of the plane of the arc
cornerRadius	Radius of the corner arc
theta	Angle between the start and end points of the arc
branch0	Path branch ID at the start of the segment
branch1	Path branch ID at the end of the segment
isLeaf	Is the segment a leaf of the path? 0 = no, 1 = yes
pathLength	The length of the entire path the segment is a part of. Used for normalizations.

AVN Parameters

Texture and Bundle Identity

Parameter	Description
`CONTENT_TYPE`	Must be `PROCEDURAL` for an AVN texture.
`WIDTH`, `HEIGHT`, `DEPTH`	Dimensions of the packed 3D texture. Physical depth includes all packed bundle layers.
`J_SCALE`	Overall AVN emission multiplier.
`K_SCALE`	Overall AVN opacity/extinction multiplier.
`MAX_GREY`, `MAX_A`	Cached texture extrema passed to the shader.
`AVN_VERSION`	V4+ uses segment headers and candidate lists.
`AVN_BUNDLE_LAYERS`	Total logical slabs packed into the physical 3D texture.
`AVN_SEGMENT_HEADER_LAYER`	logical layer containing cell candidate offsets/counts.
`AVN_CANDIDATE_LIST_LAYER`	first logical layer containing flattened candidate segment IDs.
`AVN_SEGMENT_TABLE_LAYER`	Logical layer containing fixed-stride path-segment records.
`AVN_SEGMENT_RECORD_TEXELS`	Number of texels per segment record. The current shader accesses record texels `0` through `7`.

Coordinate Transform Controls

Parameter	Description
AVN_COORDS	PATH, CARTESIAN, SPHERICAL, or CYLINDRICAL.[1]
AVN_X_ROT, AVN_Y_ROT, AVN_Z_ROT	Rotation in degrees applied to the local volume position before coordinate conversion.
AVN_X_SCALE, AVN_Y_SCALE, AVN_Z_SCALE	Scaling applied to the coordinates before noise calculation. Larger values make features broader along that axis.
AVN_X_OFFSET, AVN_Y_OFFSET, AVN_Z_OFFSET	Additive coordinate offsets before noise calculation. Good for animation..
AVN_X_PERIOD, AVN_Y_PERIOD, AVN_Z_PERIOD	Used for 4D time varying noise.
AVN_Z_FADE0	In PATH mode, where normalized path-length fade-in ends.
AVN_Z_FADE1	In PATH mode, where normalized path-length fade-out begins.

[1] Although AVN is designed for paths, noise can also be applied to regular coordinates. No special lookups are required for these other modes, simply use the ray sample world coordinates for the noise lookup.

Noise Controls

Parameter	Description
`AVN_SEED`	Integer per-texture noise seed, added to the path/coordinate seed.
`AVN_EXP`	Exponent applied after normalized noise: `pow(n, AVN_EXP)`. Higher values concentrate emission into thinner/brighter regions.
`AVN_FREQ`	Base noise frequency.
`AVN_OCTAVES`	Requested fractal-noise octave count.
`AVN_GAIN`	Amplitude multiplier for successive octaves.
`AVN_LAC`	Frequency multiplier for successive octaves.
`AVN_INVERT`	Inverts the noise function.
`AVN_TIME_FACTOR`	Multiplies animation time in the noise evaluation. `0` makes ordinary noise static.
`AVN_BACKGROUND_J`	Mix amount between noise emission and underlying base RGB emission.
`AVN_KEPLER`	Enables Keplerian angular animation in cylindrical mode when greater than zero. Higher values give “faster” rotation.